

Despliegue y configuracion

Servicios Docker

Servicio	Puerto	Funcion
mongodb	27017	Persistencia principal del sistema con MongoDB 7 y volumen mongo_data.
backend	5000	API Node/Express con healthcheck en /api/health/db.
frontend	3000	Aplicacion React servida por Nginx y proxied hacia /api.

Red entre servicios

- El stack usa la red bridge drivectrl-net.
- backend consume MongoDB mediante mongodb:27017 cuando no se inyecta DOCKER_MONGO_URI.
- frontend no llama a localhost:5000 dentro del contenedor; usa VITE_API_URL=/api y nginx.conf resuelve el proxy hacia backend.
- depends_on y healthcheck garantizan el orden minimo de arranque:
 - backend espera a mongodb
 - frontend espera a backend

Comandos para reproducir

Desde la raiz del repositorio:

```
docker compose up -d --build
docker compose ps
docker compose logs backend --tail 100
docker compose logs frontend --tail 100
curl http://localhost:5000/api/health/db
curl http://localhost:3000/
curl http://localhost:3000/api/health/db
docker compose down -v
```

Evidencia de ejecucion

Validacion del workflow Docker en GitHub Actions

Se confirmaron ejecuciones exitosas recientes del workflow docker_ci_cd.yml:

Run	Rama	Evento	Estado	URL
25774495083	main	push	success	https://github.com/puj-course/FIS_2610_3517_G4/actions/runs/25774495083
25774440079	develop	pull_request	success	https://github.com/puj-course/FIS_2610_3517_G4/actions/runs/25774440079
25774416065	develop	push	success	https://github.com/puj-course/FIS_2610_3517_G4/actions/runs/25774416065

Validacion local en este entorno

Se intentaron ejecutar estos comandos:

```
docker --version
docker compose version
docker compose up -d --build
docker compose ps
Invoke-WebRequest http://localhost:5000/api/health/db
Invoke-WebRequest http://localhost:3000/
Invoke-WebRequest http://localhost:3000/api/health/db
```

Resultado observado:

- docker --version-> Docker version 29.3.0
- docker compose version-> Docker Compose version v5.1.0
- docker compose up -d --build fallo porque el daemon local no estaba iniciado:

```
failed to connect to the docker API at
npipe:////./pipe/dockerDesktopLinuxEngine
The system cannot find the file specified.
```

- Por la misma razón, los healthchecks locales en localhost:5000 y localhost:3000 devolvieron conexión rechazada.

Healthcheck backend

El backend expone:

- GET /api/health/db
- GET /api/health/email

En Docker Compose, el servicio backend se considera sano con:

```
healthcheck:
  test: ["CMD-SHELL", "node -e
  \"fetch('http://127.0.0.1:5000/api/health/db').then(r=>process.exit(r.ok?0:1)).catch(()=>process.exit(1))\""]
```

El workflow CI/CD valida ese endpoint con:

```
curl --fail --retry 12 --retry-delay 5 --retry-all-errors
http://localhost:5000/api/health/db
```

Pipeline CI/CD

El workflow .github/workflows/docker_ci_cd.yml cubre:

1. validación de docker compose config
2. build de la imagen backend
3. build de la imagen frontend
4. docker compose up -d --build
5. validación de backend, frontend y proxy /api
6. publicación opcional a DockerHub cuando existen secretos

Publicación de imagen

La publicación está definida en el job docker-publish y usa:

- DOCKERHUB_USERNAME
- DOCKERHUB_TOKEN

Imágenes configuradas:

- \${DOCKERHUB_USERNAME}/drivetr1-backend:\${github.sha}
- \${DOCKERHUB_USERNAME}/drivetr1-backend:latest

- `${DOCKERHUB_USERNAME}/drivectl-frontend:${github.sha}`
- `${DOCKERHUB_USERNAME}/drivectl-frontend:latest`

Si los secretos no existen, el workflow no falla; simplemente omite el push y deja traza en logs.

Variables de entorno necesarias

Backend

Archivo base: `backend/.env.example`

Variables principales:

Variable	Uso
MONGO_URI	Conexion completa a MongoDB para backend y Compose.
JWT_SECRET	Firma de tokens JWT.
GOOGLE_CLIENT_ID	Validacion de Google Auth en backend.
EMAIL_HOST, EMAIL_PORT, EMAIL_USER, EMAIL_PASS	SMTP para OTP y recuperacion.
TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_PHONE_NUMBER	Fallback SMS opcional.
OTP_EXPIRACION_MINUTOS, OTP_MAX_INTENTOS, OTP_COOLDOWN_SEGUNDOS	Políticas del flujo OTP.

Frontend

Archivo base: `apps/web/.env.example`

Variable	Uso
VITE_API_URL	URL base consumida por el frontend.
VITE_GOOGLE_CLIENT_ID	Client ID publico usado por Google Login en frontend.

Problemas encontrados y solucion

Problema	Impacto	Solucion aplicada
backend/.env.example tenia valores sensibles reales	Riesgo de exponer credenciales en el repositorio	Se reemplazaron por placeholders seguros y documentados.
El daemon local de Docker no estaba activo en este entorno	No fue posible levantar docker compose ni capturar healthchecks locales aqui	Se dejo trazabilidad exacta del error y evidencia de que el workflow CI/CD si valida el stack en GitHub Actions.
La publicacion a DockerHub depende de secretos	Sin secretos no hay push de imagenes desde Actions	El workflow ya contempla este caso y lo omite sin romper el pipeline.

Estado de entrega de la parte de despliegue

- Dockerfile backend funcional
- Dockerfile frontend funcional
- Docker Compose definido para frontend, backend y MongoDB
- Healthchecks declarados
- Workflow Docker CI/CD validado en GitHub Actions
- .env.example sanitizado y documentado
- Evidencia local con docker compose up en esta maquina (bloqueado por daemon de Docker apagado)
- Evidencia de DockerHub (requiere secretos configurados y captura del repositorio)